

Dependency Injection (DI) & Inversion of Control (IoC)

1 What is Inversion of Control (IoC)?

Normally, application code creates and manages its own dependencies.

Without IoC: `UserService` controls dependency creation directly.

```
class UserService {  
    private UserRepository repository = new UserRepository();  
}
```

With Spring IoC: Object creation and dependency management are transferred from the application code to the Spring container.

```
@Service  
class UserService {  
    private final UserRepository repository;  
  
    UserService(UserRepository repository) {  
        this.repository = repository;  
    }  
}
```

2 What is Dependency Injection (DI)?

DI is the specific mechanism through which dependencies are provided to a class by the framework.

IoC (Spring controls object creation/management) → **DI** (Spring injects required dependencies)

Here, `UserService` depends on `UserRepository`, and Spring injects `UserRepository` into `UserService`.

3 Types of Dependency Injection

RECOMMENDED Constructor Injection:

```
@Service
public class UserService {
    private final UserRepository repository;

    public UserService(UserRepository repository) {
        this.repository = repository;
    }
}
```

Advantages: Dependencies are explicit, supports `final` (immutability), improves unit testability, and prevents instantiating incomplete objects.

OPTIONAL Setter Injection:

```
@Service
public class UserService {
    private UserRepository repository;

    @Autowired
    public void setRepository(UserRepository repository) {
        this.repository = repository;
    }
}
```

Use case: Useful when dependencies are optional or changeable at runtime.

NOT PREFERRED Field Injection:

```
@Service
public class UserService {
    @Autowired
    private UserRepository repository;
}
```

Problems: Harder to unit test (requires reflection), dependencies are hidden, cannot use `final` fields.

4 How Spring Performs Dependency Injection

Component Scanning → Find `UserRepository` → Create `UserRepository` Bean → Find `UserService`
→ Resolve Dependency → Inject → Create `UserService` Bean

5 Interface + Dependency Injection (DIP)

Depending on abstractions rather than concrete implementations promotes loose coupling and adheres to SOLID's **Dependency Inversion Principle**.

```
public interface PaymentService { void pay(); }

@Service
public class UpiPaymentService implements PaymentService { }

@Service
public class OrderService {
    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

6 Resolving Ambiguity with Multiple Implementations

If multiple beans match an interface type (e.g., `UPIPaymentService` and `CardPaymentService`), Spring throws a `NoUniqueBeanDefinitionException` unless resolved using:

Option A: `@Qualifier` (Targeted Selection)

```
public OrderService(@Qualifier("upiPaymentService") PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

Option B: `@Primary` (Default Candidate)

```
@Primary
@Service
public class UpiPaymentService implements PaymentService { }
```

INTERVIEW ANSWER

"If multiple beans match the required type, Spring can resolve ambiguity using `@Primary` (to set a default candidate) or `@Qualifier` (to explicitly specify the bean name)."

7 The IoC Container

Spring's IoC container is responsible for bean creation, lifecycle management, dependency injection, and scope resolution. The primary container abstraction is `ApplicationContext`.

8 IoC vs. DI — Core Distinction

Dimension	Inversion of Control (IoC)	Dependency Injection (DI)
Nature	Design Principle / High-Level Concept	Design Pattern / Implementation Mechanism
Core Role	Control of object creation/management is inverted	Dependencies are actively injected into the object
Scope	Broader architectural principle	Specific way to implement IoC

BEST INTERVIEW ANSWER

"IoC is the principle of transferring control of object creation and dependency management to the framework. Dependency Injection is the mechanism Spring uses to provide a class with its required dependencies."

9 Key Cross-Questions & Solutions

Q1. Why is constructor injection preferred?

It makes dependencies explicit, supports immutability via `final` fields, improves testability without Spring, and prevents incomplete object creation.

Q2. What is loose coupling in DI?

It means high-level components depend on interfaces (abstractions) rather than instantiating concrete implementations (`new ServiceImpl()`), allowing implementations to swap easily.

Q3. What exceptions occur when dependency injection fails?

Missing dependency → `NoSuchBeanDefinitionException`.

Multiple candidates without qualifier/primary → `NoUniqueBeanDefinitionException`.

Q4. Is `@Autowired` mandatory for constructor injection?

No. As of Spring 4.3+, if a class has a single constructor, Spring implicitly uses it for DI without requiring `@Autowired`.

Q5. How does DI relate to SOLID principles?

It directly realizes the **Dependency Inversion Principle (DIP)** and supports the **Single Responsibility Principle (SRP)** by removing creation logic from business classes.

Master Cheat Sheet & Key Summary

- **IoC:** Spring controls object creation & lifecycle.
- **DI:** Spring provides required dependencies to objects.
- **Constructor Injection:** Best practice approach.
- **@Qualifier vs. @Primary:** Name-specific lookup vs. Default bean selection.
- **ApplicationContext:** The central IoC container abstraction.

One Sentence to Remember:

"IoC is the principle; DI is the mechanism; Spring's ApplicationContext is the container that creates and manages the beans and injects their dependencies."